

Colocation

Live demo: `/colocation` · Code: `src/app/colocation/`

What it is

Because **only** `page` and `route` files become public URLs, you can safely keep related files — components, helpers, tests, styles, even this guide — **in the same folder** as the page that uses them. They will never accidentally become a route.

Files in this folder

File	Routable?	Purpose
<code>colocation/page.tsx</code>	→ <code>/colocation</code>	The page
<code>colocation/_ui/Card.tsx</code>		A component used by the page
<code>colocation/helpers.ts</code>		A util used by the page
<code>colocation/card.test.ts</code>		A colocated test
<code>colocation/GUIDE.md</code> / <code>.pdf</code>		This guide

Note `helpers.ts` has **no** underscore and is **not** a special file, yet it's still not a URL — that's colocation. The `_ui` folder additionally uses the private-folder convention.

Why it's safe

A route segment becomes public **only** when it contains `page.js` / `route.js`, and only the content those files return is sent to the client. Everything else is just project code.

Organization strategies (from the docs)

1. Keep all shared code **outside** `app` (e.g. in `src/components`, `src/lib`).
2. Keep shared code at the **top of** `app`.
3. **Split by feature**: shared code at the app root, route-specific code colocated in the route folder (*This lab uses #3.*)

Pick one and be consistent.

Interview Q&A

- **Q: Can I put a `utils.ts` next to my `page.tsx` without it becoming a route?** Yes. Only `page` / `route` files are routable; everything else is safe to colocate.
- **Q: Does a folder with a `page.tsx` expose its other files as URLs?** No — only the `page` / `route` content is served.